CSA1006 – SOFTWARE ENGINEERING

Name: Purushothaman.V

Reg no.: 192511064

Unit 4 Assignment

Question 2: CI/CD Pipeline and Docker Containerization

Introduction

CI/CD (Continuous Integration and Continuous Delivery/Deployment) is a software development practice that automates the process of building, testing, and delivering software. Tools such as **Jenkins** and **GitHub Actions** can automatically execute these activities whenever developers push new code.

Docker supports this process by packaging an application along with its dependencies into a **container**, helping the application run consistently across development, testing, and production environments. The question specifically asks about the key CI/CD stages and how Docker supports continuous delivery.

1. Key Stages of a CI/CD Pipeline

1. Source Code Management

Developers write code and upload their changes to a version-control repository such as GitHub.

Example:
A developer pushes a new feature to a GitHub repository.

Developer → Git Repository

The CI/CD system detects the new change and starts the pipeline.

2. Build

The pipeline downloads the required dependencies and builds the application.

For example:

- Install project dependencies
- Compile source code if required
- Create the application package

If the build fails, the pipeline stops and the developer is notified.

3. Automated Testing

Automated tests are executed to check whether the new code works correctly.

Common tests include:

- Unit testing
- Integration testing
- API testing
- Regression testing

If a test fails, the code is not promoted to the next stage until the problem is fixed.

4. Code Quality and Security Checks

The pipeline can perform automated checks for:

- Coding errors
- Code quality
- Vulnerabilities
- Dependency issues

This helps identify problems before the application reaches production.

5. Package and Containerize

After successful testing, the application can be packaged into a Docker image.

The Docker image contains:

- Application code
- Required libraries
- Runtime environment
- Configuration needed to run the application

This creates a consistent application package.

6. Deployment

The validated application is deployed to a staging or production environment.

A typical approach is:

Successful Tests

↓

Staging Environment

↓

Final Verification

↓

Production Environment

Depending on the organization's setup, deployment can be fully automatic or require approval before production release.

7. Monitoring and Feedback

After deployment, the application is monitored to identify failures or performance problems.

The collected feedback can be used by developers to fix problems and improve future releases.

2. CI/CD Pipeline Flow

Developer

|

▼

Git Repository

|

▼

Build Application

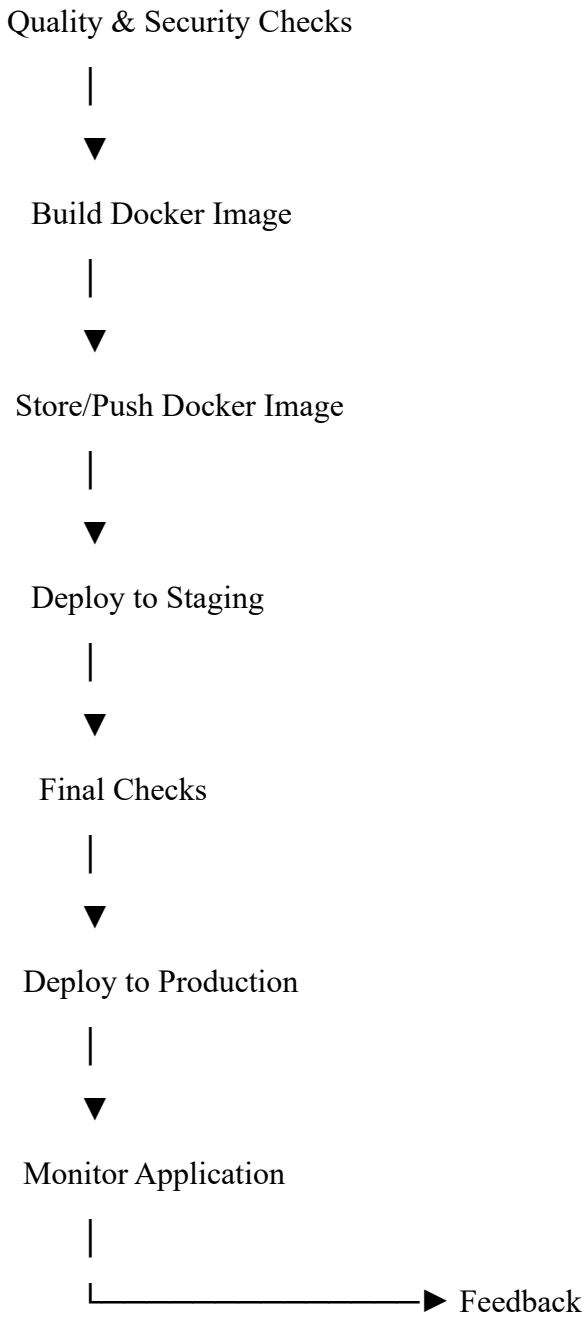
|

▼

Automated Testing

|

▼



3. Role of Jenkins or GitHub Actions

Jenkins or **GitHub Actions** can act as the automation engine for the CI/CD pipeline.

For example, when a developer pushes code:

1. The CI/CD tool detects the change.
2. Dependencies are installed.
3. The application is built.
4. Automated tests are executed.

5. Docker image is created.
6. The image can be pushed to a container registry.
7. The application is deployed to the required environment.

This reduces the amount of manual work involved in software delivery.

4. How Docker Supports Continuous Delivery

Docker improves continuous delivery by creating a standardized environment for running an application.

1. Environment Consistency

The same Docker image can be used during testing and deployment.

Development → Testing → Production

Same Docker Image

Therefore, differences between development and production environments are reduced.

2. Dependency Management

All required libraries and runtime dependencies can be packaged into the Docker image.

This prevents problems such as:

"It works on my machine, but not on the server."

3. Faster Deployment

Containers can be started quickly, allowing applications to be delivered and updated efficiently.

4. Easy Rollback

A previous Docker image can be redeployed if a new version causes problems.

Version 1 → Version 2 → Problem

↓

Rollback

↓

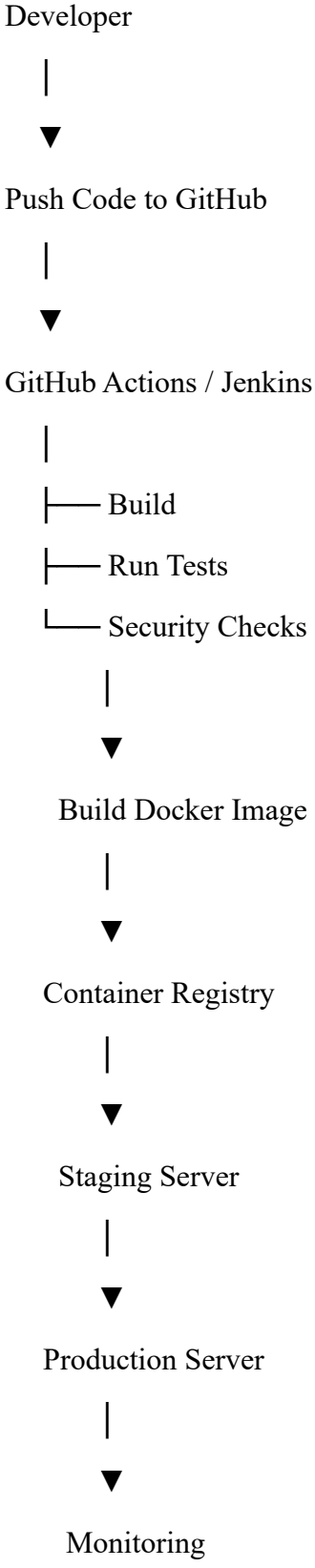
Version 1

5. Portability

Docker containers can run across different environments that support Docker, making it easier to move applications between development, testing, and production systems.

5. Example of CI/CD with Docker

Consider an online shopping application.



If all automated checks succeed, the Docker image can be promoted for deployment. If problems are detected, the pipeline can stop before production deployment.

Conclusion

A **CI/CD pipeline** automates the major stages of software delivery, including source-code integration, building, testing, quality checking, packaging, deployment, and monitoring. **Jenkins or GitHub Actions** can automate these stages and provide rapid feedback to developers. Docker strengthens continuous delivery by packaging applications and their dependencies into portable containers, providing consistency across environments, simplifying deployment, and supporting reliable version management and rollback. Together, CI/CD and Docker enable software teams to deliver updates **faster, more consistently, and with fewer deployment-related problems**.